Accelerating Quantized 2D Image Convolution on Apple Silicon via AMX-Accelerated im2col–GEMM Lowering

Abstract In computer vision applications, a main computational expense is two-dimensional spatial convolution, whose nested loops can suffer from poor cache locality. The im2col transformation unfolds the sliding convolutional patches into a dense matrix and reduces convolution to a general matrix multiplication (GEMM). We introduce a quantized 2D image convolution implementation based on Apple's undocumented AMX (Apple Matrix Coprocessor) using tile-local im2col–GEMM lowering. We describe the AMX register model, its memory layout, the considerations needed to expose data reusability to the 32 × 32 outer-product engine, and the quantization used for inputs and weights. A source-level audit identifies that periodic draining in the original implementation was a consequence of selecting 16-bit accumulation, rather than an unavoidable restriction of AMX: the revised kernel selects native 32-bit accumulation and handles spatial and channel tails exactly. On Apple M3, we evaluate six kernel sizes, five image resolutions, channel and reduction-length sweeps, one to four worker threads, and complete pretrained ResNet-18 inference. At the three original resolutions, the revised AMX kernel achieves 1.45–1.52× speedup over our NEON baseline for 3 × 3 and 1.49–1.55× for 5 × 5. Complete quantized ResNet-18 inference takes 23.88 ± 0.13 ms with one worker and 10.44 ± 0.23 ms with four, compared with 80.61 ± 0.36 ms and 31.63 ± 0.28 ms for the same graph using NEON.

Keywords: Apple AMX, im2col, GEMM, quantized convolution, ARM NEON, Apple Silicon

Introduction

Direct 2D convolution involves loops over the output channels, spatial location, and the receptive field of the filter. This access pattern requires suitable packing or tiling before it can efficiently use fixed-tile matrix operations. A standard solution is to unfold sliding patches into the rows of a dense $(N \times K)$ matrix, where $N = H_{\text{out}}W_{\text{out}}$ and $K = C_{\text{in}}K_hK_w$; flatten the filter to $(C_{\text{out}} \times K)$; and reduce convolution to Eq (1):

$$Y = \text{im2col}(X)W^{\mathsf{T}}. \tag{1}$$

This approach sacrifices the additional memory required for im2col but gains reuse of tuned GEMM kernels [5,6]. Anderson et al. demonstrate that im2col is only one of several possible lowering strategies [2]; later work improves the transformation itself [7,27] or uses FFT-based alternatives [24]. Winograd and Cook–Toom implementations provide another alternative to im2col-based convolution on mobile ARM CPUs [14].

In this paper, we assume the im2col–GEMM lowering in Eq (1) and ask a more focused question: is AMX a good target for the resulting GEMM when packing and output retrieval are included? GPU tensor-core benchmarking demonstrates substantial GEMM acceleration while emphasizing the importance of memory-hierarchy management [15]. Related work examines tensor-core programmability and the relationship between precision and performance in dense matrix computations [16]. Comparative CPU/GPU matrix-multiplication benchmarks further illustrate how acceleration depends on problem size and execution platform [3]. AMX is an undocumented, integrated CPU matrix engine on Apple Silicon. Its instruction set is known through community reverse engineering [12]. Prior applications include Saber/FrodoKEM key encapsulation [8], NTRU polynomial multiplication [9], and workload-independent microarchitectural characterization [26]. These studies motivate an application-level evaluation, rather than a claim that the present experiments exhaustively characterize the hardware.

Convolution using NEON on ARM CPUs is well studied, including 16-bit NEON assembly evaluated against OpenCV and the ARM Compute Library [19], and direct-convolution packing/compute overlap on ARM multicores [22]. NEON performance has also been evaluated through on-board benchmarks on space-grade ARM processors [4]. A second, orthogonal motivation for a fixed-point kernel is work on unified INT8 training [28], distribution-adaptive INT8 training [25], and integer CNN inference for embedded object detection [23]. Quantization-aware optimization also addresses precision allocation and distribution mismatch to balance inference efficiency and accuracy [1]. Other work addresses quantization-aware training on edge devices through coreset selection and cascaded layer correction [21]. These deployment questions apply to MobileNet [10], YOLO [17], and residual networks [30]. The channel counts and operator mixes of these networks differ substantially.

Simonyan and Zisserman showed that stacked 3×3 convolutions can achieve the receptive field of larger filters with fewer parameters and FLOPs [29]. As a result, 3×3 convolutions have become common in vision architectures.

Methodology

It is computationally expensive to naively compute direct convolution. This section explains how this paper reduces convolution to GEMM, map the resulting computation to AMX, and establish a benchmark to evaluate this approach.

The im2col–GEMM reduction

Fig 1 illustrates the transformation end to end. The CPU unfolds patches into a spatial tile, which is then multiplied by prepared filters to produce the output feature map. Given $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ and $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}$, the conceptual patch matrix has N rows and K columns. The implementation stores the input in CHW order and the output in spatial-row/channel (NC) order. For each tile, it gathers at most 32 output positions into a $(K \times 32)$ INT16 buffer. Spatial padding and unused positions are zeroed. Filters are packed into $(K \times 32)$ channel tiles once, when a prepared plan is created. Thus, Fig 1 implements Eq (1) without materializing the complete patch matrix.

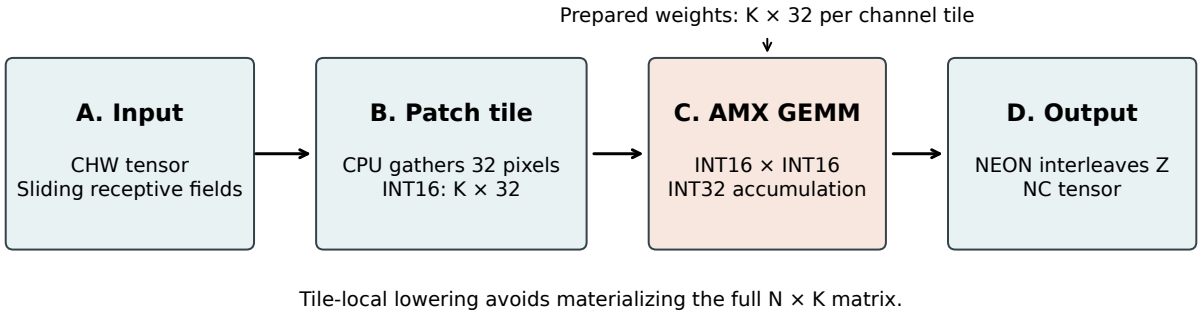


Fig 1. im2col lowering from patches to output.

AMX architecture and register model

AMX has three fixed-width register files. Table 1 summarizes their physical storage and the mode used here; the interpretation of their lanes depends on the selected instruction mode [12]. The relevant instructions are

Table 1. AMX registers.

File	Rows	Bytes per row	Use in this kernel
X	8	64	32 INT16 weight operands
Y	8	64	32 INT16 patch operands
Z	64	64	32×32 INT32 results

`AMX_SET` (enable state), `AMX_LDX`/`AMX_LDY` (load operands), `AMX_MAC16` (outer-product accumulation), `AMX_STZ` (store a physical Z row), and `AMX_CLR` (disable state). The revised matrix-mode operand sets bit 62 to select INT32 Z accumulation, whereas the original operand left this bit clear. Each reduction step computes up to 1,024 useful MACs for a full tile. This value is an operation count per instruction, not a measured issue rate per clock cycle. INT32 results occupy interleaved physical rows; NEON zip instructions restore consecutive output-channel order after all Z stores have been issued.

Each AMX worker is responsible for enabling and disabling its own state. Accumulators are initialized by omitting the previous Z input from the first MAC, rather than by treating `AMX_CLR` as an accumulator-zeroing instruction. Workers request `QOS_CLASS_USER_INTERACTIVE`; this is a scheduling hint, not CPU affinity or proof of execution on a particular core [32]. The evaluation does not rely on a claim that scheduling on an efficiency core necessarily produces `SIGILL`.

Quantization scheme

Model activations and BN-folded filter weights use symmetric linear quantization. Eq (2) defines the scale for a tensor or weight channel, with $S_x = 1$ for an all-zero tensor:

$$S_x = \frac{\max |x|}{127} \quad (\max |x| > 0). \tag{2}$$

Eq (3) rounds to the nearest value, with ties to even, and clips the result to the symmetric range:

$$x_q = \mathrm{clip}(\mathrm{round}_{\mathrm{even}}(x/S_x), -127, 127)\,. \tag{3}$$

Model activations use a dynamic per-tensor scale computed at each layer. After batch-normalization folding, weights use one scale per output channel. The synthetic correctness and convolution benchmarks also cover the full signed INT8 range $[-128, 127]$. Inputs and weights are sign-extended to the INT16 operands used by AMX, whereas inner products accumulate directly in INT32. Rescaling applies the product of the activation and weight-channel scales, followed by the folded bias. Integer accumulation is exact within the bound established below, but agreement with an integer reference does not imply zero quantization error relative to the original floating-point model.

Bounds for Accumulation and Retrieving Results

The risk of accumulator overflow is determined by the magnitudes of the input operands and kernel weights, as well as the reduction length. Let $A = \max |x_q|$ denote the maximum absolute input value and $B = \max |w_q|$ the maximum absolute kernel weight. Without assuming cancellation, Eq (4) bounds the magnitude of a sum of q products by qAB. Consequently, the maximum number of terms, q_b, that can be safely accumulated in a signed b-bit integer is:

$$\left| \sum_{k=1}^{q} x_{q,k} w_{q,k} \right| \leq qAB, \qquad q_b = \left\lfloor \frac{2^{b-1} - 1}{AB} \right\rfloor. \tag{4}$$

For instance, with $A = B = 127$, as in symmetric 8-bit quantization, Eq (4) guarantees that only two terms ($q_{16} = 2$) can be safely accumulated in a signed 16-bit integer (INT16). This means that neither a reduction length of $K = 27$ nor $K = 75$ can be safely processed in a single INT16 accumulation pass for all possible inputs. Full-range signed 8-bit integers (INT8), which range from -128 to 127, make the limit stricter. A single product of $(-128)(-128)$ equals 16,384; adding another such product gives 32,768, exceeding the INT16 maximum of 32,767. The safe accumulation limit is therefore reduced to one term ($q_{16} = 1$), highlighting a correctness flaw in the original fixed two-term batching approach.

For full-range INT8 inputs and weights, Eq (5) establishes the safe reduction-length limit for the revised kernel, which accumulates in signed 32-bit integers (INT32):

$$K \leq \left\lfloor \frac{2^{31} - 1}{128^2} \right\rfloor = 131{,}071. \tag{5}$$

The prepared-plan API rejects reductions exceeding this limit to prevent signed overflow. All tested layers satisfy this requirement. The revised AMX path performs the entire reduction in native INT32 Z registers and retrieves the final result only once. A separate, explicitly labeled INT16 ablation retrieves each product and resets the accumulator before the next multiplication, ensuring exact results even for operand values of -128.

Fig 2 illustrates data reuse in the outer-product calculation. A dense tile spanning 32 spatial positions and 32 output channels performs 1,024 useful multiply-accumulate operations per reduction step, whereas a tile with 32 spatial positions and a single output channel performs only 32. Channel and spatial tails are padded with zeros for AMX computation, and only valid results are retained.

Comparison Benchmarks and Parallel Processing

Our standard C implementation, compiled with the `-O3` optimization flag, serves as a baseline. Its contiguous channel loop allows automatic vectorization by the compiler, so it is designated as a C baseline rather than a simple scalar implementation. The NEON baseline widens INT16 products and accumulates them in INT32 vectors. All computational paths share a common strategy for gathering tile-local patches and arranging prepared weights, keeping the initial processing stages consistent while differing in their core computation loops. This comparison evaluates AMX against these established baselines, not against unoptimized scalar code.

A prepared plan allocates weights, scratch buffers and creates a persistent pthread pool before timed inference. Each worker processes separate spatial tiles and output regions while sharing read-only weights. Single-worker plans run on the calling thread. Meanwhile multiworker plans include pool wake-up, synchronization, and contention.

Hardware and benchmark methodology

The revised experiments were performed on an Apple M3 (Mac15,12) with four performance and four efficiency cores, 16 GB unified memory, and a reported 128 KiB performance-core L1 data cache. The rerun environment is macOS 27.0 (26A428) and Apple Clang 21.0.0, with `-O3 -ffast-math`; it is distinct from the original macOS 15/Clang 16 environment. The old timing tables are retained only in the supplied original manuscript and are not combined with the new data.

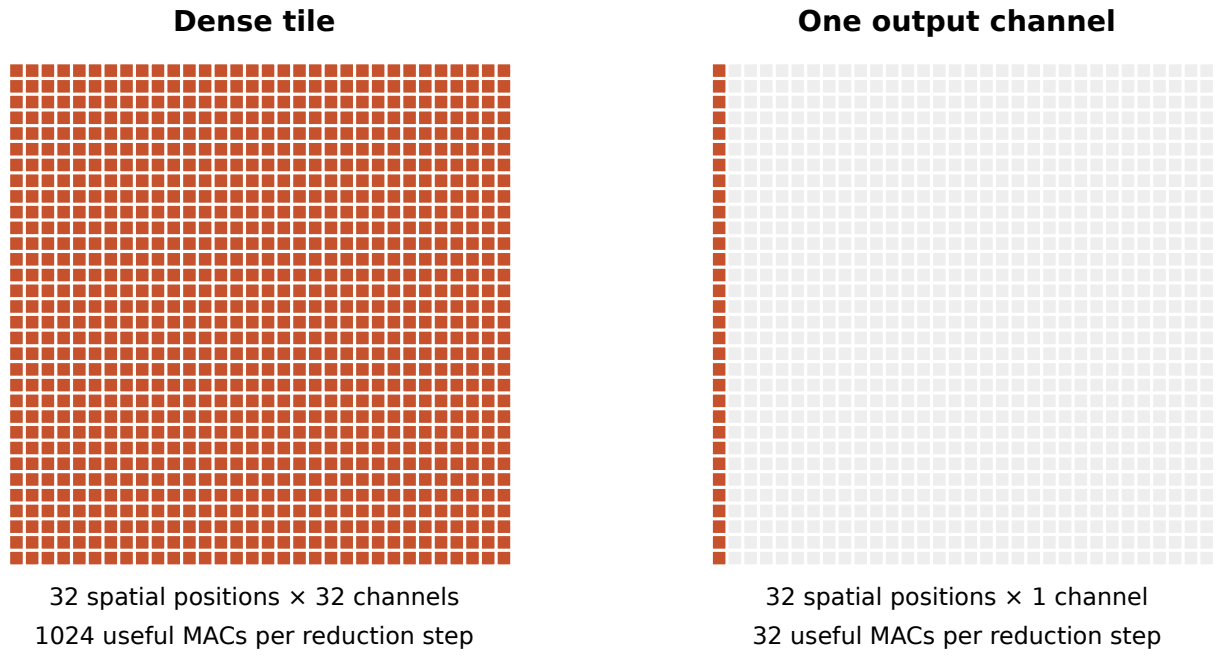


Fig 2. AMX outer-product data reuse.

Each backend/configuration receives five unrecorded warm-ups and 30 timed calls using `clock_gettime(CLOCK_MONOTONIC)`. Backend order rotates across trials. These are sequential samples in one measurement session, not 30 independent machines or sessions. Memory-clobber barriers surround timing boundaries, outputs are consumed, and outputs are compared before the convolution sweep. Warm-up does not guarantee that all working sets fit in cache or that clock frequency is constant. We report the sample mean and sample standard deviation, retaining all measured samples, including those from high-variance configurations.

Eq (6) defines useful throughput by counting one multiplication and one addition for each valid convolution MAC, excluding operations on padded tile elements:

$$\mathrm{GOPS} = \frac{2NC_{\mathrm{out}}K}{\bar{t} \times 10^6}, \qquad \bar{t} \text{ in milliseconds.} \tag{6}$$

Eq (7) defines the speedup of AMX relative to the corresponding baseline backend for the same shape and worker count:

$$\mathrm{Speedup} = \frac{\bar{t}_{\mathrm{baseline}}}{\bar{t}_{\mathrm{AMX}}}. \tag{7}$$

The calculation of convolution metrics includes the time spent packing patches, computing matrix products, and retrieving outputs. Plan creation and weight packing are excluded and amortized over repeated inference; cold-start costs are not measured. Phase profiling is performed separately after the timed trials, so timer calls inside tiles do not inflate the principal latency measurements. Phase times across workers are sums of worker durations and must not be treated as elapsed wall time.

Design space and model evaluation

Table 2 lists the controlled sweeps. This is a set of spatial/kernel combinations and one-factor channel sweeps, not a full Cartesian search over every dimension. Including thread scaling and the drain ablation gives 263 backend/configuration combinations and 7,890 timed convolution calls. We also execute the complete pretrained torchvision ResNet-18 (`IMAGENET1K_V1`) [30, 31], including its 20 convolutions, residual

Table 2. Design space.

Sweep	Values and fixed dimensions
Spatial/kernel	$H = W \in \{32, 64, 128, 224, 256\}$; $K_h = K_w \in \{1, 3, 5, 7, 9, 11\}$; $C_{\text{in}} = 3$, $C_{\text{out}} = 32$
Input channels	$C_{\text{in}} \in \{1, 3, 8, 16, 32, 64\}$; 64×64, 3×3, $C_{\text{out}} = 32$
Output channels	$C_{\text{out}} \in \{1, 8, 16, 24, 31, 32, 33, 64, 128\}$; 64×64, 3×3, $C_{\text{in}} = 16$
Reduction length	24 values from $K = 1$ to 512, including 26–28, 31–33, 63–65, and 74–76; 16×16, 1×1, $C_{\text{out}} = 32$
Worker count	$p \in \{1, 2, 3, 4\}$ on four shapes, including 56×56, $64 \to 64$, 3×3 and 28×28, $64 \to 128$, stride two
Drain ablation	64×64, $C_{\text{in}} = 3$, $C_{\text{out}} = 32$; 3×3 and 5×5; one worker

additions, ReLUs, max pooling, global average pooling, and classifier. Batch normalization is folded into filters and bias before quantization. Input is one recorded, normalized 224×224 RGB photograph from the PyTorch tutorial's hymenoptera validation set. The same prepared graph uses C, NEON, or AMX convolution, with one to four workers per convolution. Timed inference includes dynamic activation quantization, rescaling, tensor-layout conversion, and nonconvolution operators; model loading, preprocessing, and plan creation are excluded. The classifier and nonconvolution operators remain C/FP32. The evaluation targets batch-one latency of the complete graph with quantized convolutions and FP32 supporting operators.

Correctness and instrumentation

Google Test compares each backend with an independent direct-convolution INT64 reference over 144 shapes and 12 backend/worker combinations per shape (1,728 combinations, each invoked twice). These cover channel tails, spatial tails, stride one/two, multiple kernels, and full-range INT8. Additional tests cover all--128 products and invalid/unsafe dimensions. All five test cases pass, including an AddressSanitizer/UndefinedBehaviorSanitizer build. AMX inline-assembly memory accesses are additionally checked by output sentinels; compiler sanitizers do not instrument coprocessor stores themselves.

Instrumentation combines elapsed-time profiling, source-level traffic analysis, and sampled CPU power. Default Instruments counter traces are archived; named cache-miss counts, measured DRAM bandwidth, and exact AMX cycles are not reported because the required event mapping was not verified. For power collection, `powermetrics` supplied CPU-subsystem power estimates at 250 ms intervals. We retain 60 workload samples and 20 preceding idle samples per run for three repetitions of each NEON/AMX and one/four-worker combination, except the one-worker NEON configuration, for which two complete runs remain. Only complete retained records contribute to the reported estimates.

Eq (8) specifies how the supplied power workflow reports estimated CPU-subsystem energy for the collected samples:

$$E_{\text{call}} = \frac{\overline{P}_{\text{CPU}} T}{R}, \qquad E_{\text{inc}} = \frac{(\overline{P}_{\text{CPU}} - \overline{P}_{\text{idle}}) T}{R}, \qquad E_{\text{GOP}} = \frac{E_{\text{inc}}}{2 N C_{\text{out}} K / 10^9}. \tag{8}$$

In Eq (8), R is the number of repeated calls, T their elapsed time, and average power is time-weighted over approximately 15.18 s overlapping each 16 s workload run. The per-call conversion uses throughput over the full workload run and therefore assumes that the sampled window is representative of that run. The protocol uses three repeats, paired idle samples, and one/four-worker NEON and AMX runs of a fixed convolution. These are estimated CPU-subsystem quantities, not a dedicated AMX rail or whole-device energy measurement. Energy improvement is not inferred from reduced latency.

Performance Evaluation

We first report convolution-call latency at the three original resolutions, then examine the wider design space, parallel scaling, complete-model performance, and output retrieval costs. All figures and tables below are generated from the archived raw trials of the revised code.

Overall latency and throughput

Table 3 and Table 4 report mean ± sample SD in milliseconds for same-padded stride-one convolution with $C_{\text{in}} = 3$ and $C_{\text{out}} = 32$. Their speedups use Eq (7). Fig 3 and Fig 4 extend the comparison to all five resolutions, with throughput calculated using Eq (6). At 64×64, AMX takes 0.167 ms for 3×3, compared

Table 3. 3×3 latency.

Resolution	C (ms)	NEON (ms)	AMX (ms)	vs. C	vs. NEON
64×64	0.470 ± 0.015	0.241 ± 0.008	0.167 ± 0.001	2.82×	1.45×
128×128	1.926 ± 0.020	0.977 ± 0.013	0.649 ± 0.003	2.97×	1.50×
256×256	9.802 ± 2.862	5.203 ± 1.904	3.425 ± 1.250	2.86×	1.52×

Table 4. 5×5 latency.

Resolution	C (ms)	NEON (ms)	AMX (ms)	vs. C	vs. NEON
64×64	1.311 ± 0.027	0.611 ± 0.020	0.411 ± 0.002	3.19×	1.49×
128×128	5.359 ± 0.029	2.453 ± 0.055	1.618 ± 0.003	3.31×	1.52×
256×256	21.425 ± 0.026	9.873 ± 0.042	6.368 ± 0.019	3.36×	1.55×

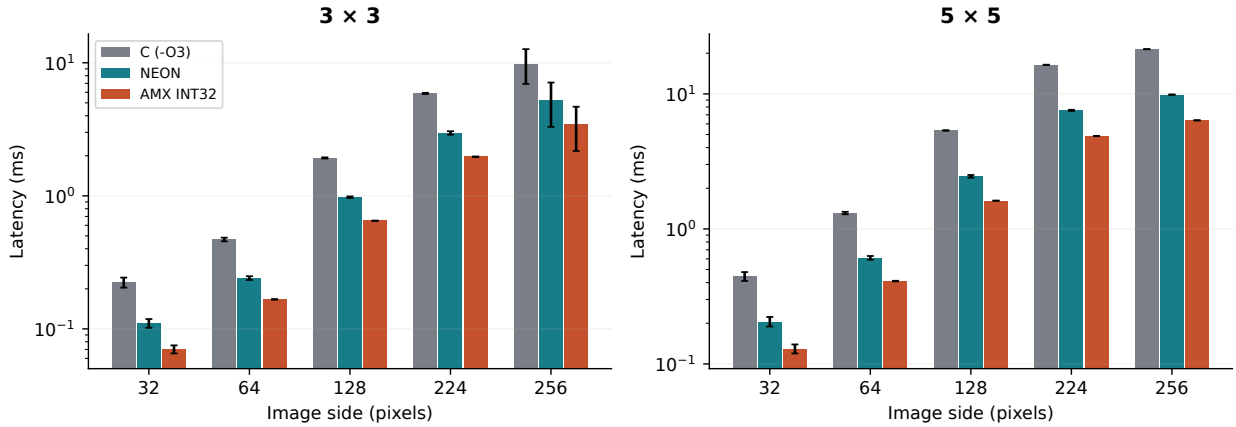


Fig 3. Convolution latency across image resolutions.

with 0.241 ms for NEON and 0.470 ms for C. At 128×128, the corresponding values are 0.649, 0.977, and 1.926 ms (Table 3). AMX's mean speedup over NEON is 1.45–1.52× across the original resolutions. The 256×256 3×3 measurements have substantial variability for all three backends; the reported AMX mean is 3.425 ± 1.250 ms. This variability is retained rather than attributed to cache misses without counters.

For 5×5, the revised AMX kernel is faster than both supplied baselines at the three original resolutions: 0.411, 1.618, and 6.368 ms (Table 4), giving 1.49–1.55× speedup over NEON. Thus the former $K = 75$ loss is not an unavoidable AMX architectural limit. These results supersede the old timing tables; changed accumulation, packing, baseline loops, compiler, and operating system prevent interpreting the entire difference as the effect of one code edit.

Kernel, channel, and reduction scaling

Fig 5 plots the measured AMX speedup over C and NEON across six kernel sizes. Fig 6 independently varies input and output channels. They show why kernel size alone is an insufficient dispatch rule. At fixed 64×64, 3×3, and $C_{\text{in}} = 16$, AMX achieves approximately 0.93× NEON speedup for one output channel and 3.09× for 128 output channels (Fig 6). This is consistent with reuse across output-channel tiles and the underutilized case in Fig 2. Depending on channel tails and the compiler path taken, the measured curve may be nonmonotonic. A smooth theoretical curve should therefore not be substituted for the measured data.

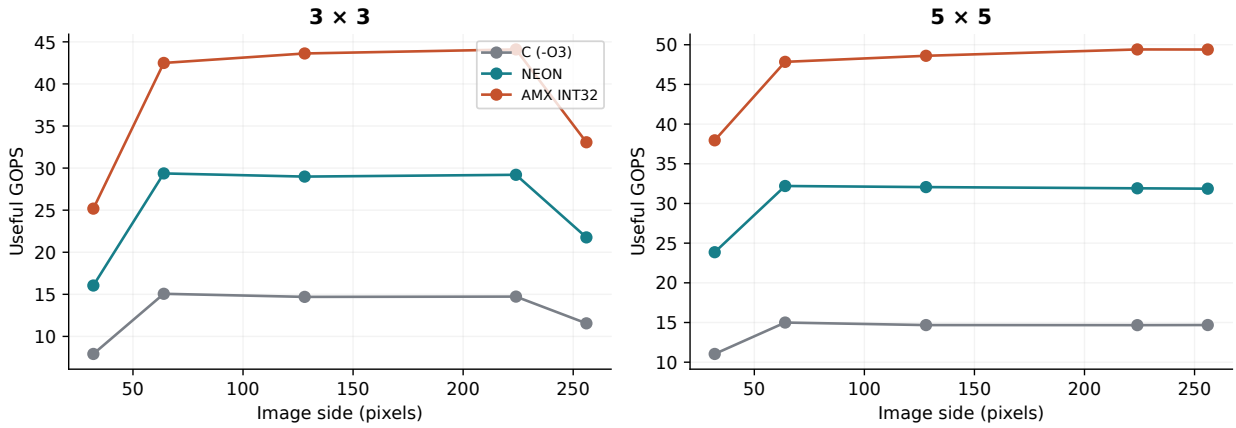


Fig 4. Convolution throughput across image resolutions.

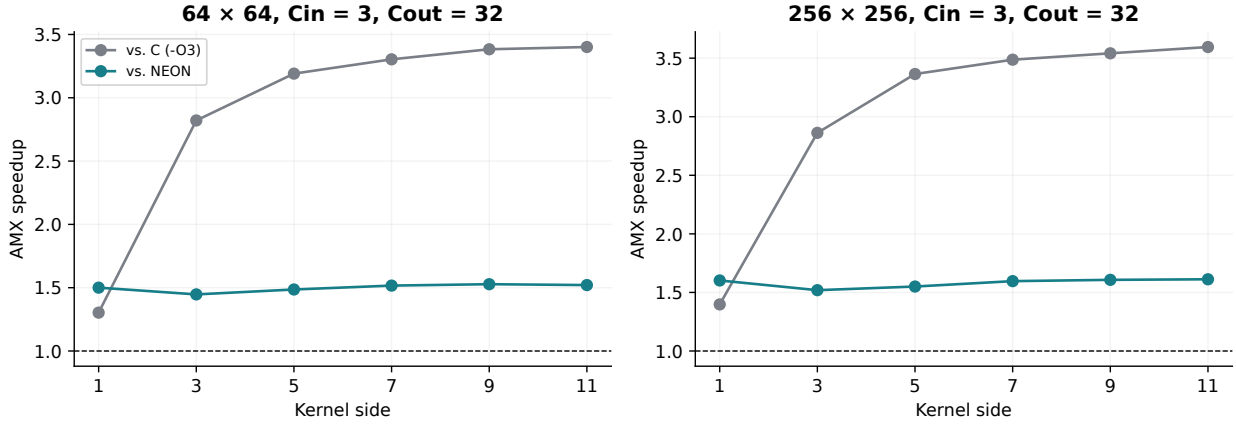


Fig 5. Latency scaling across kernel sizes.

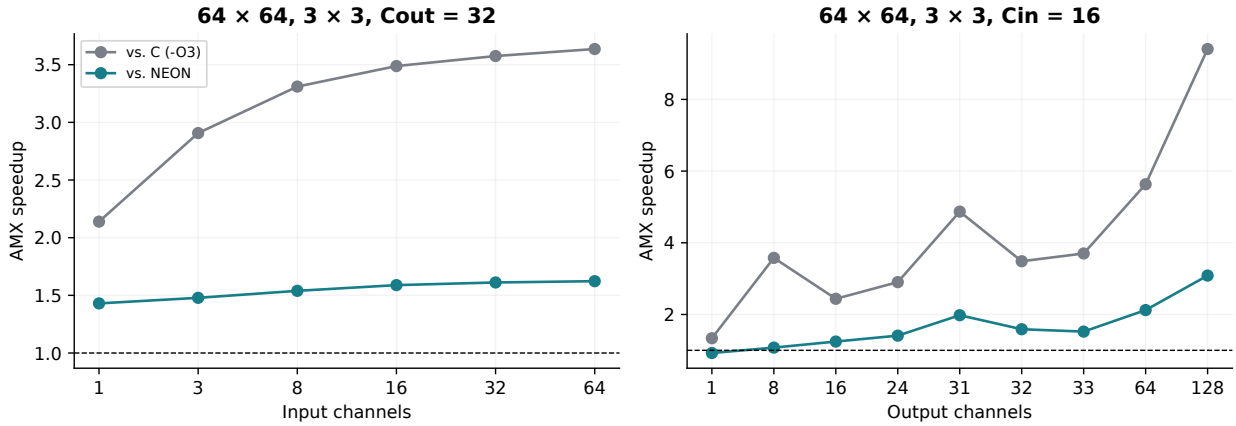


Fig 6. Performance scaling across channel counts.

Fig 7 shows 24 reduction lengths, including points immediately around 27 and 75. The native INT32 path has no drain transition at either value. Its AMX/NEON speedup is approximately 1.43–1.74× in this sweep. Eq (5), rather than a $K = 27/K = 75$ distinction, determines the supported reduction range.

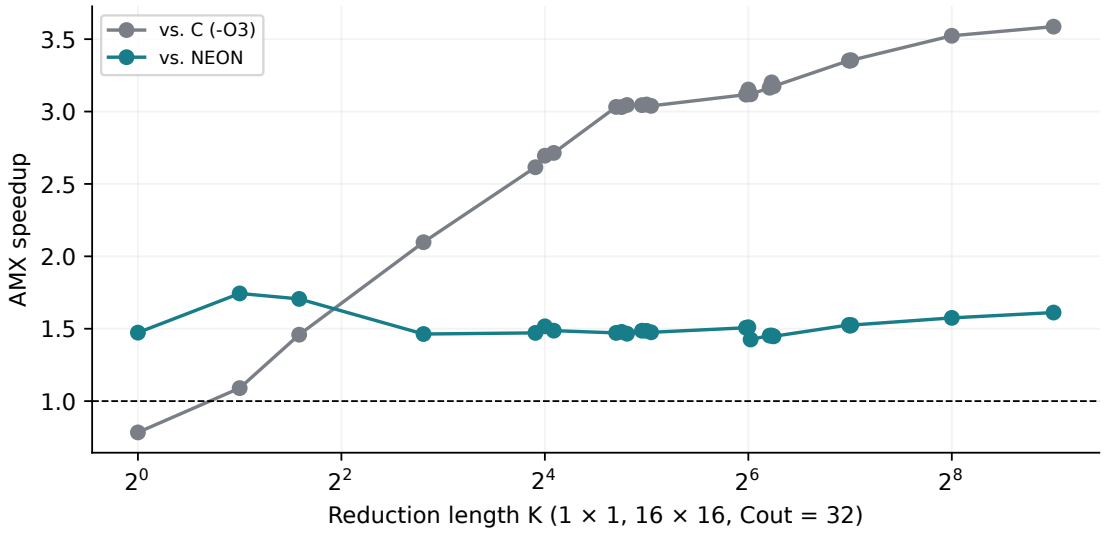


Fig 7. AMX performance across reduction lengths.

Multithreaded scaling

Fig 8 reports scaling at the same shape relative to one worker. Eq (9) defines scaling and parallel efficiency, including synchronization and shared-resource contention:

$$S_p = \frac{\bar{t}_1}{\bar{t}_p}, \qquad \eta_p = \frac{S_p}{p}. \tag{9}$$

For the 56×56, $64 \to 64$, 3×3 shape, AMX decreases from 2.251 ms to 0.634 ms with four workers:

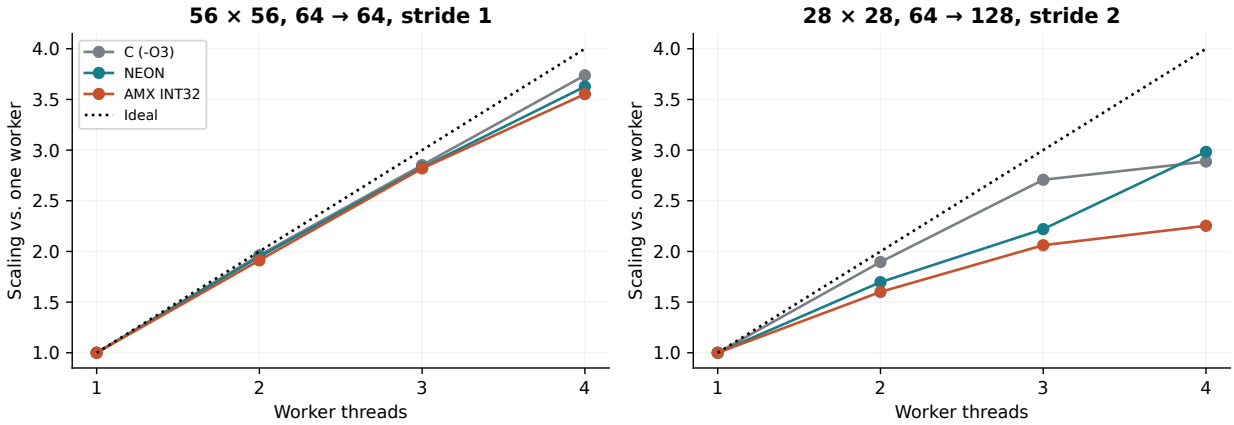


Fig 8. Convolution speedup with multiple workers.

$S_4 = 3.55$ and $\eta_4 = 88.8\%$ by Eq (9). For 28×28, $64 \to 128$, stride two, AMX decreases from 0.158 ms to 0.070 ms: $S_4 = 2.25$ and $\eta_4 = 56.3\%$. The smaller amount of parallel work makes coordination relatively more expensive. These application measurements demonstrate useful concurrent execution on this machine; they do not establish one independent AMX engine per core, absence of contention, or scaling beyond four workers.

Complete ResNet-18 inference

Table 5 and Fig 9 report full graph latency. AMX takes 23.88 ± 0.13 ms with one worker and 10.44 ± 0.23 ms with four, versus 80.61 ± 0.36 ms and 31.63 ± 0.28 ms for NEON. The respective model-level speedups are $3.38\times$ and $3.03\times$, using Eq (7). AMX's four-worker model scaling is $2.29\times$, or 57.2% parallel efficiency using Eq (9); it is substantially lower than the large-layer scaling and must not be described as linear. The

Table 5. ResNet-18 latency.

Workers	C (ms)	NEON (ms)	AMX (ms)	vs. NEON
1	203.36 ± 4.20	80.61 ± 0.36	23.88 ± 0.13	$3.38\times$
2	114.56 ± 0.55	46.30 ± 0.16	14.88 ± 0.10	$3.11\times$
3	88.68 ± 0.32	37.10 ± 0.20	11.84 ± 0.05	$3.13\times$
4	77.83 ± 0.49	31.63 ± 0.28	10.44 ± 0.23	$3.03\times$

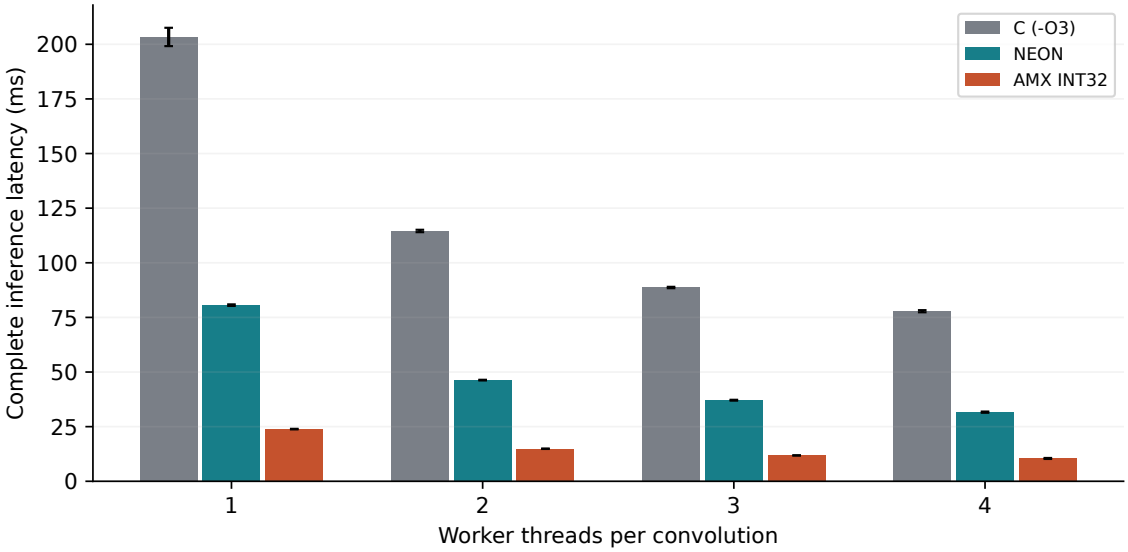


Fig 9. Complete quantized ResNet-18 inference latency.

quantized graph produces identical output logits across backends and worker counts. On the recorded photograph, both the FP32 model and the quantized graph select class 410 (apiary); the maximum absolute logit difference from FP32 is 0.346. This is a numerical sanity check on one input, not an accuracy estimate. The larger model-level speedup relative to the three-channel synthetic cases is consistent with reuse in deeper, wider layers. Depthwise MobileNet operators, YOLO graphs, batch sizes above one, and complete production inference frameworks are not evaluated here.

Drain cost and logical traffic

Fig 10 separates loop-control time, 32 INT16 Z-row stores, INT16 store/load/widen/add, and final INT32 store/interleave/write. The hot-buffer microbenchmark performs 10,000 repeated 32×32 tile operations per timing sample, with 30 samples per mode; error bars show sample SD. Loop control costs 1.80 ± 0.59 ns per tile, INT16 stores 26.61 ± 22.08 ns, complete INT16 draining 146.17 ± 53.64 ns, and final INT32 retrieval 174.06 ± 55.93 ns. These values are elapsed amortized times, not the latency of an individual instruction or an exact count of hardware cycles. The sizeable SD prevents precise comparisons between individual store instructions. Table 6 enumerates the logical data movement for one full tile, excluding input/weight loads, initial output zeroing, and cache-line effects. Each INT16 drain stores 2,048 Z bytes, reads those bytes on the CPU, and reads/writes 4,096 output bytes. Native INT32 retrieval stores and reads 4,096 bytes once and writes the output once. These values are source-level byte counts rather than measurements of DRAM traffic.

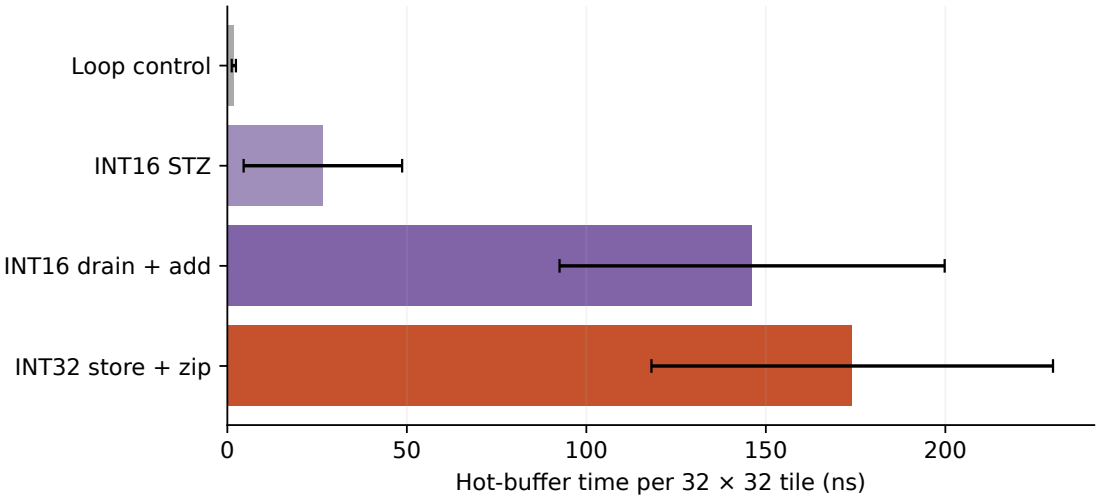


Fig 10. INT16 drain and INT32 retrieval costs.

Eq (10) expresses the counts in Table 6. The old two-term path is included for source analysis and is not

Table 6. Drain traffic.

Path	Transfers per tile	$K = 27$ bytes	$K = 75$ bytes
Old INT16, two-term batches	$\lceil K/2 \rceil$	172,032	466,944
Safe INT16, one-term batches	K	331,776	921,600
Native INT32	1	12,288	12,288

guaranteed correct on full-range INT8:

$$D = \lceil K/q_{16} \rceil, \qquad Q_{16} = 12{,}288D, \qquad Q_{32} = 12{,}288 \quad \text{bytes per full tile.} \tag{10}$$

In the separately timed, correctness-preserving convolution ablation at 64×64, AMX INT16 with one-term draining takes 0.578 ms versus 0.164 ms for native INT32 at $K = 27$, and 1.606 ms versus 0.412 ms at $K = 75$. This supports the importance of drain elimination at both reductions. Eq (10) predicts the traffic increase, but an exact cycle cost requires named counters and cannot be inferred by multiplying nanoseconds by an assumed maximum clock.

Packing and cache interpretation

Separate AMX phase profiling at 64×64 gives approximately 0.135 ms for packing and 0.032 ms for compute/output retrieval at $K = 27$, and 0.382 ms versus 0.034 ms at $K = 75$. The respective packing fractions of these instrumented phase sums are about 81% and 92%. Thus AMX acceleration does not make the whole lowering pipeline compute-bound: CPU gathering dominates these cases.

Eq (11) distinguishes full materialization from tile-local storage, excluding input/output tensors and extraction scratch:

$$S_{\text{full}} = NK \text{ bytes (INT8)}, \qquad S_{\text{tile}} = 64K \text{ bytes per worker}, \qquad S_W = 64K \lceil C_{\text{out}}/32 \rceil \text{ bytes.} \tag{11}$$

For $K = 27$ and 75, Eq (11) gives only 1,728 and 4,800 bytes of patch scratch per worker; this does not grow with resolution. AMX extraction additionally uses up to 6 KiB of stack scratch per worker plus a 128-byte tail row. The original argument based on a full $N \times K$ matrix exceeding L1 therefore does not describe the revised tile-local implementation. Larger tensors still change input/output footprints, but no causal L1-miss or bandwidth claim is made without a verified counter measurement.

Estimated CPU power and energy

Table 7 and Fig 11 report time-weighted estimated CPU power and energy per call, using Eq (8). Each entry is mean ± sample SD across three complete measurement runs (two for one-worker NEON), rather than the SD of the 60 within-run power samples. The fixed workload has $H = W = 128$, $C_{\text{in}} = C_{\text{out}} = 64$, and a same-padded 3×3 filter: 1.208 useful GOP per call. These measurements concern that convolution, not complete ResNet-18 energy. With one worker, estimated total CPU energy per call is 63.09 ± 6.60 mJ for

Table 7. CPU energy.

Workers	Backend	Runs	CPU W	mJ/call	Incremental mJ/call
1	NEON	2	6.36 ± 1.19	170.56 ± 34.32	35.21 ± 134.38
1	AMX	3	5.26 ± 0.51	63.09 ± 6.60	42.86 ± 20.66
4	NEON	3	16.17 ± 1.20	123.77 ± 8.59	118.74 ± 11.60
4	AMX	3	14.85 ± 0.79	59.97 ± 12.05	53.60 ± 8.17

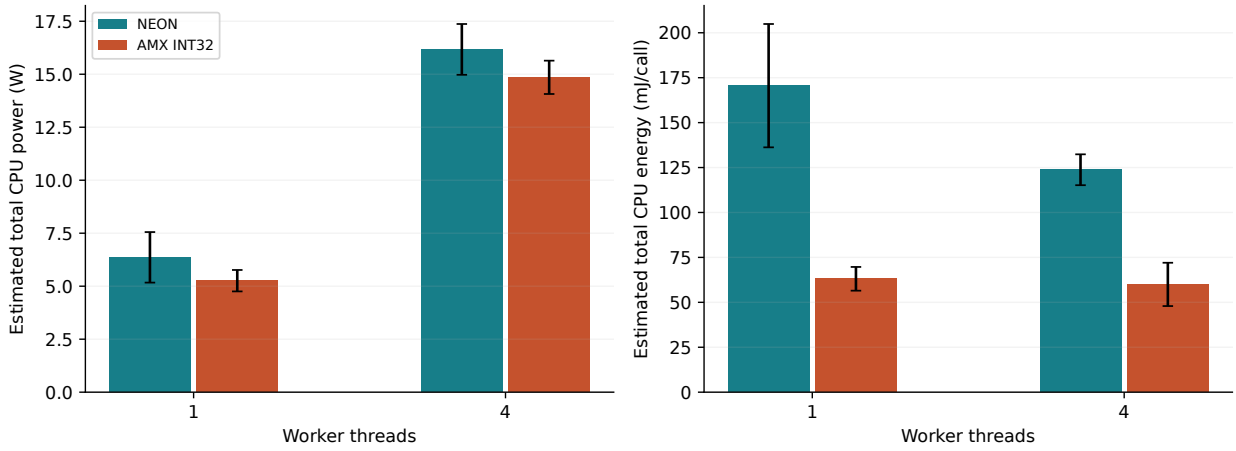


Fig 11. Estimated CPU power and energy per call.

AMX and 170.56 ± 34.32 mJ for NEON; with four workers it is 59.97 ± 12.05 mJ and 123.77 ± 8.59 mJ. The observed total-energy ratios are 2.70× and 2.06×. Mean CPU power is lower for AMX in these runs, but four-worker execution consumes more power than one-worker execution for both backends. The CPU sensor includes background CPU activity and may not expose all energy attributable to AMX.

Idle-subtracted estimates require additional care. One one-worker NEON run recorded 9.42 W during the preceding idle window and 7.20 W during its workload, giving a negative incremental estimate. We preserve it in the raw data; the one-worker NEON incremental mean consequently has an SD larger than the mean. Four-worker mean incremental energy is 53.60 ± 8.17 mJ/call for AMX and 118.74 ± 11.60 mJ/call for NEON, or 0.0444 ± 0.0068 and 0.0983 ± 0.0096 J/GOP by Eq (8).

Interpretation and Implications

Resolving the mixed result

The inconsistent performance observed stems from the INT16 accumulation mode and a rigid batching strategy that necessitated frequent data flushes. As demonstrated in Equations 4 and 5, a clear boundary free from draining does not exist between $k = 27$ and $k = 75$. The reduction sweep directly validates this: by switching to native INT32 accumulation, this transitional behavior disappears. The prior performance drop was an implementation quirk, not an inherent limitation of AMX architecture.

Eq (12) presents a functional model for performance. 'L' represents the time spent lowering data, 'T' (calculated as the ceiling of $N/32$ multiplied by the ceiling of $C_{\text{out}}/32$) denotes the number of output tiles. 'a'

is the effective cost per loaded MAC operation, 'd' is the cost of retrieving output, and 'h_p' accounts for overheads related to coordination and contention. These are empirically measured values that fluctuate based on matrix dimensions and the number of active workers, rather than raw instruction cycle counts.

The equations for INT16 and INT32 accumulation times are:

$$\begin{aligned} t_{16} &\approx L + T(Ka_{16} + Dd_{16}) + h_p, \\ t_{32} &\approx L + T(Ka_{32} + d_{32}) + h_p. \end{aligned} \tag{12}$$

This clarifies why draining after each small batch can negate the benefits of matrix reuse. Once the costs associated with data lowering and coordination are distributed, Eq (13) offers a simple criterion for determining when AMX outperforms a baseline system with tile costs of $Ka_B + d_B$:

$$\text{AMX proves superior if} \qquad K(a_B - a_{32}) > d_{32} - d_B. \tag{13}$$

This condition is derived analytically, not through empirical fitting. The actual costs are influenced by factors such as effective channel utilization, spatial occupancy, cache performance, instruction dependencies, and scheduling. The instrumentation used in this study does not pinpoint a precise clock-cycle threshold. However, the channel sweep does offer a practical foundation for optimizing dispatch on this particular hardware. A rigid guideline dictating a 3×3 versus 5×5 approach would be inconsistent with the updated findings.

Why AMX targets the multiplication

This work focuses on applying matrix hardware to the standard im2col–GEMM formulation rather than introducing a new lowering method [6]. The approach can complement methods that reduce im2col storage or generate patches on demand [2, 7, 27]. AMX is useful when each patch vector contributes to enough output channels to amortize patch-gathering and output-retrieval costs. It does not perform the im2col copying operation itself. The complete-model result shows a system-level latency benefit for this ResNet-18 implementation, while the phase measurements identify CPU packing as the next target for optimization.

For a kernel-only upper-bound model, Eq (14) combines useful work on a full tile with the AMX operand loads and final logical output traffic:

$$I_{\text{tile}} = \frac{2(32)(32)K}{128K + 12{,}288} \quad \text{operations per logical byte.} \tag{14}$$

Eq (14) excludes patch construction, cache-line allocation, and transfers beyond the local hierarchy. It is therefore neither a measured DRAM arithmetic intensity nor a roofline classification. As Fig 2 shows, underutilized channel tiles reduce the number of useful operations without proportionally reducing AMX work. The $C_{\text{out}} = 1$ result is consequently an important counterexample to an unconditional AMX speedup claim.

Evaluation scope

Table 8 summarizes the evaluation scope. The experiments backing this paper are supported by independent correctness tests, drain measurements, analytical bounds on traffic, and CPU energy estimates. The results are sourced from execution on Apple M3 chip. Extending these findings to other hardware generations, inference frameworks, platforms, and model families requires further evaluation.

Future work

The next steps are named-cache/cycle instrumentation and repeated CPU-energy collection using Eq (8), with better control of background activity. These measurements should be followed by repeated sessions under recorded thermal and frequency conditions. Natural implementation improvements include reducing patch-gathering overhead, measuring plan-creation costs, and adding packed-input reuse. As expressed by Eq (13), dispatch should depend on shape, occupancy, and measured costs rather than on a presumed

Table 8. Evaluation scope.

Metric	Evaluation basis
Convolution latency	30 timed calls per backend/configuration; common prepared lowering
Parallel execution	One to four workers; shape-level and full-graph scaling
Model latency	Complete pretrained ResNet-18; one recorded real input
Correctness	Independent INT64 convolution reference; exact backend/model agreement
Drain cost	Hot-buffer nanoseconds and convolution ablation; analytical logical bytes
Cache/DRAM	Analytical working sets and logical traffic; named misses and measured DRAM bandwidth not reported
Energy/power	Two/three complete estimated CPU-power runs per configuration; background activity limits idle subtraction
Exact AMX cycles	Elapsed-time drain profiling; instruction-cycle counts not reported

accumulator-overflow threshold at $K = 75$. Depthwise/separable MobileNet layers [10], YOLO backbones [13, 17], and accuracy validation should then be used to test how far the present ResNet-18 result extends. Extensions to sparse workloads would require attention to sparse storage formats, irregular accesses, and accelerator-specific data movement [11]. Beyond CNN inference, ZeroQAT investigates memory-efficient on-device quantization-aware training for language models [20].

Conclusion

We presented a quantized 2D convolution implementation using tile-local im2col–GEMM lowering and native AMX INT32 accumulation on Apple M3. The revised implementation eliminates avoidable INT16 drains, corrects full-range overflow and tail handling, and provides persistent parallel workers with a common lowering strategy across the baselines. Wider kernel, channel, and reduction sweeps show that the earlier $K = 27/K = 75$ explanation did not represent an architectural boundary. At the original resolutions, AMX outperforms the supplied NEON baseline for both kernel sizes. Complete pretrained ResNet-18 inference improves by 3.38× with one worker and 3.03× with four workers. We believe that the proposed methodology can provide a faster and energy-efficient baseline for future im2col–GEMM convolutions on Apple Silicon hardware.

Credit Authorship Contribution Statement

Diljot Singh: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing: original draft. **Omana J:** Conceptualization, Methodology, Supervision, Validation, Writing: review & editing. **Indira B:** Conceptualization, Validation, Visualization, Writing: review & editing. All authors have read and approved the final version of the manuscript.

Data Availability

The code and diagrams related to the AMX-accelerated im2col–GEMM convolution implementation are available at `https://github.com/lqSky7/AMX_IM2COL`.

Declaration of Competing Interests

The authors declare that they have no conflict of interest.

Funding

No external funding was received for this research.

Acknowledgement

The authors acknowledge Vellore Institute of Technology, Chennai, for its support of this research.

References

1. M. Almoudane, "A quantization-aware optimization framework for efficient deep neural network inference," *Int. J. Res. Innov. Soc. Sci.*, vol. 10, pp. 2141–2156, 2026.

2. A. Anderson, A. Vasudevan, C. Keane, and D. Gregg, "High-performance low-memory lowering: GEMM-based algorithms for DNN convolution," in *SBAC-PAD*, 2020. doi: 10.1109/SBAC-PAD49847.2020.00024.

3. M. Q. Ansari and M. Q. Ansari, "Accelerating matrix multiplication: A performance comparison between multi-core CPU and GPU," arXiv:2507.19723, 2025.

4. M. Blanco, "On-board performance benchmark (OBPMark) of Arm NEON SIMD on space-grade processors," in *EDHPC*, 2025.

5. K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *IWFHR*, 2006.

6. S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, et al., "cuDNN: Efficient primitives for deep learning," arXiv:1410.0759, 2014.

7. J. Fornt, P. Fontova-Musté, M. Caro, J. Abella, F. Moll, J. Altet, et al., "An energy-efficient GeMM-based convolution accelerator with on-the-fly im2col," *IEEE Trans. VLSI Syst.*, vol. 31, pp. 1874–1878, 2023. doi: 10.1109/TVLSI.2023.3286122.

8. D. L. Gazzoni Filho, G. Brandão, G. Adj, A. Alblooshi, I. A. Canales-Martínez, J. Chávez-Saab, et al., "PQC-AMX: Accelerating Saber and FrodoKEM on the Apple M1 and M3 SoCs," in *ARITH*, 2024. doi: 10.1109/ARITH61404.2024.00018.

9. D. L. Gazzoni Filho, G. Brandão, and J. López, "Fast polynomial multiplication using matrix multiplication accelerators with applications to NTRU on Apple M1/M3 SoCs," *IACR Commun. Cryptol.*, vol. 1, 2024. doi: 10.62056/a3txommol.

10. A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

11. V. Isaac-Chassande, A. Evans, Y. Durand, and F. Rousseau, "Dedicated hardware accelerators for processing of sparse matrices and vectors: A survey," *ACM Trans. Archit. Code Optim.*, vol. 21, art. 27, 2024. doi: 10.1145/3640542.

12. D. Johnson and P. Cawley, "Apple AMX instruction set," GitHub repository, 2022. `https://github.com/corsix/amx`. Accessed Oct. 1, 2026.

13. M. Kotthapalli, D. Ravipati, and R. Bhatia, "YOLOv1 to YOLOv11: A comprehensive survey," arXiv:2508.02067, 2025.

14. P. Maji, A. Mundy, G. Dasika, G. Beu, M. Mattina, and R. Mullins, "Efficient Winograd or Cook–Toom convolution kernel implementation on mobile CPUs," in *EMC^2 Workshop, HPCA*, 2019. doi: 10.48550/arXiv.1903.01521.

15. M. Martineau, P. Atkinson, and S. McIntosh-Smith, "Benchmarking the NVIDIA V100 GPU and tensor cores," in *Euro-Par 2018 Workshops*, LNCS 11339, 2018. doi: 10.1007/978-3-030-10549-5_35.

16. R. Piñán García, *Targeting High-Performance Applications with NVIDIA Tensor Cores*. Master's thesis, Technical University of Munich, 2024.

17. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *CVPR*, 2016. doi: 10.1109/CVPR.2016.91.

18. S. A. C. Romero and S. E. M. Guerra, "INT8 quantization makes ARM edge inference dispatch-invariant," arXiv:2607.23227, 2026.

19. A. Shevchenko, P. Prystavka, and V. Tymchyshyn, "A SIMD-based approach to the enhancement of convolution operation performance," *CEUR Workshop Proc.*, vol. 2588, 2019.

20. Q. Tan, X. Song, N. Cheng, G. Li, J. Liu, L. Hong, et al., "ZeroQAT: End-to-end on-device quantization-aware training for LLMs at inference cost," arXiv preprint, 2025.

21. Y. Tong, J. Yuan, and C. Hu, "Enhancing quantization-aware training on edge devices," arXiv:2507.17768, 2025.

22. P. Wang, W. Yang, J. Fang, D. Dong, C. Huang, P. Zhang, et al., "Optimizing direct convolutions on ARM multi-cores," in *SC23*, 2023. doi: 10.1145/3581784.3607068.

23. P. Xiao, C. Zhang, Q. Guo, X. Xiao, and H. Wang, "Neural networks integer computation: Quantizing CNNs for object detection in embedded systems," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, 2024. doi: 10.1109/JSTARS.2024.3452321.

24. Y. Zhang and X. Li, "Fast convolutional neural networks with fine-grained FFTs," in *PACT '20*, pp. 255–265, 2020. doi: 10.1145/3410463.3414642.

25. K. Zhao, S. Huang, P. Pan, Y. Li, Y. Zhang, Z. Gu, et al., "Distribution adaptive INT8 quantization for training CNNs," in *AAAI*, 2021. doi: 10.1609/aaai.v35i4.16462.

26. J. Zhou, *Performance Analysis of the Apple AMX Matrix Accelerator*. Bachelor's thesis, Massachusetts Institute of Technology, 2025.

27. Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, et al., "Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators," in *IISWC*, 2021. doi: 10.1109/IISWC53590.2021.00030.

28. F. Zhu, R. Gong, F. Yu, X. Liu, Y. Wang, Z. Li, et al., "Towards unified INT8 training for convolutional neural networks," in *CVPR*, 2020. doi: 10.1109/CVPR42600.2020.00199.

29. K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *ICLR*, 2015. arXiv:1409.1556.

30. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016. arXiv:1512.03385.

31. PyTorch contributors, "ResNet-18," *Torchvision model documentation*. `https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html`. Accessed Oct. 1, 2026.

32. Apple, "Optimize for Apple Silicon with performance and efficiency cores," 2020. `https://developer.apple.com/news/?id=vk3m204o`. Accessed Oct. 1, 2026.